Studio Nx

JS 33 — initSearch()

2026-10-12

JS 33 — `initSearch()` (+ `refreshList`)

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 검색과 정렬 (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	32 `renderPager()` · 34 `initRowMenus()`

1. 하는 일

목록 위의 검색 칸이나 정렬 칸이 바뀌면 **1쪽부터 목록을 다시 받아서 그립니다**.

사용자가 한 일	언제 요청하나	결과 (주소)
검색 칸에 `alp`를 입력	입력을 **멈추고 0.3초 뒤에 한 번**	`alpine, Alpha Song` (`?query=alp`)
공백을 하나 더 입력	요청하지 않습니다 (검색어가 같음)	그대로
정렬을 Name으로 바꿈	**바로**	`Alpha Song, alpine` (`?query=alp&sort=title`)
검색 칸을 비움	0.3초 뒤	전체 4개 (`?sort=title`)
`gam`을 입력하고 바로 Enter	**바로, 한 번만** (기다리던 요청은 취소)	`Gamma Loop`

함께 만든 것

- `refreshList(options)`: "받기(30) → 목록(31) → 페이지 줄(32)"의 세 줄을 한 함수로 묶었습니다. 32번 페이지 넘김도 이것을 쓰도록 바꿨고, 35번과 37번도 씁니다.
- **30번 `loadProjects`에 두 줄 추가: 늦게 도착한 옛 응답 버리기.** 아래 §5를 참고하십시오.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initSearch();`로 검색 칸과 정렬 칸에 이벤트 처리를 한 번 걸어 둡니다

디바운스(debounce)는 글자를 칠 때마다 요청하지 않고, **입력이 멈춘 뒤** 정해진 시간이 지나면 한 번만 요청하는 방법입니다. 칠 때마다 이전 타이머를 지우고(`clearTimeout`) 새로 걸기(`setTimeout`) 때문에 마지막 타이머만 남습니다. `alp`를 0.06초 간격으로 치면 요청은 3번이 아니라 1번만 갑니다.

3. 코드

import (`myproject.js` 맨 위: 30번과 같고, 이번에 추가한 것은 없습니다)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
```

```
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import를 쓰지 않습니다. 같은 파일의 `current`, `loadProjects`, `renderProjects`, `renderPager`를 씁니다.

상수 (이번에 추가)

```
/** 검색 칸 입력을 멈춘 뒤 기다리는 시간 (ms) */
const SEARCH_DELAY = 300;

/** loadProjects 요청 번호 — 늦게 도착한 옛 응답을 버리려고 */
let loadTicket = 0;
```

함수

```
export function initSearch() {
  const form = document.getElementById('project-search');
  const input = document.getElementById('project-query');
  const sort = document.getElementById('project-sort');
  if (!form || !input || !sort) return;
  let timer = null;

  const search = function () {
    clearTimeout(timer);
    refreshList({ query: input.value, sort: sort.value, page: 1 });
  };

  input.addEventListener('input', function () {
    clearTimeout(timer);
    if (input.value.trim() === current.query) return;   // 공백만 더 쳤거나 원래대로 되돌림
    timer = setTimeout(search, SEARCH_DELAY);
  });
  sort.addEventListener('change', search);
  form.addEventListener('submit', function (event) {
    event.preventDefault();   // 페이지를 새로 열지 않음
    search();
  });
}

/** 목록 다시 받아 그리기 — loadProjects → renderProjects → renderPager (실패 · 옛 응답이면 아무것도 안
함) */
async function refreshList(options) {
  const data = await loadProjects(options);
  if (!data) return null;
  renderProjects(data);
  renderPager(data.total, data.page, data.size);
  return data;
}
```

30번 `loadProjects`에 추가한 부분

```
  const ticket = ++loadTicket;
  let data;
  try {
    data = await api.get('/projects?' + params.toString());
```

```
  } catch (err) {
    if (ticket !== loadTicket) return null;   // 더 새 요청이 있으면 이 실패는 무시
    …
  }
  if (ticket !== loadTicket) return null;   // 그사이 더 새 요청을 보냄 → 이 응답은 버림 (33)
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
input.addEventListener('input', () => { … });
form.addEventListener('submit', (event) => { event.preventDefault(); search(); });
```

최신 문법	실제 코드	뜻
`() => { … }`	`function () { … }`	함수입니다

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 함수를 부릅니다

식	뜻	예
`setTimeout(search, SEARCH_DELAY)`	0.3초 뒤에 `search`를 한 번 부르고, 타이머 번호를 돌려줍니다	
`clearTimeout(timer)`	아직 불리지 않은 타이머를 취소합니다 (이미 지났거나 null이면 아무 일도 없습니다)	다음 글자를 치면 앞의 타이머를 취소합니다
`input.value.trim() === current.query`	지금 목록의 검색어와 같으면 요청할 필요가 없습니다	`'alp '`와 `'alp'`는 같습니다
`'input'`, `'change'`, `'submit'` 이벤트	차례대로 글자가 바뀔 때마다, select 값을 고른 뒤, Enter(폼 보내기)	
`event.preventDefault()`	폼의 기본 동작(주소에 `?query=`를 붙여 페이지를 새로 열기)을 막습니다	
`++loadTicket`	1을 늘린 **뒤의** 값입니다. 요청마다 새 번호가 됩니다	1, 2, 3 …
`ticket !== loadTicket`	내가 보낸 뒤에 다른 요청이 또 나갔는지 확인합니다	내 번호가 1이고 지금이 2이면 옛 응답입니다

4. 흐름

- **글자를 입력할 때:** 타이머를 취소하고, 검색어가 지금과 같으면 끝냅니다. 다르면 0.3초 타이머를 겁니다.
- **정렬을 바꾸거나 Enter를 누를 때:** 타이머를 취소하고 바로 `refreshList({ query, sort, page: 1 })`를 부릅니다.
- **`refreshList`:** 목록을 받고, 실패했거나 옛 응답이면 `null`로 끝냅니다. 아니면 목록과 페이지 줄을 그립니다.

5. 왜 이렇게

선택	이유
0.3초 디바운스	템플릿과 HTML 주석에 적힌 값입니다. 칠 때마다 요청하면 서버에 부담이 되고 목록이 깜빡입니다

선택	이유
정렬과 Enter는 바로 요청	한 번 고르는 동작이라 기다릴 이유가 없습니다. Enter는 "지금 찾아"라는 뜻입니다
검색어가 같으면 건너뜀	공백만 더 쳤거나, 지웠다가 다시 친 경우에 같은 결과를 또 받지 않습니다
검색과 정렬은 항상 1쪽부터	3쪽을 보다가 검색하면 결과가 1쪽뿐일 수 있습니다
옛 응답 버리기 (30번에 추가)	`beta` 요청의 응답이 늦게 도착하면, 그 뒤에 보낸 `alpine`의 결과를 덮어써서 화면과 주소가 `beta`로 돌아갑니다. 그래서 요청마다 번호를 붙이고, 마지막 번호의 응답만 씁니다. **시험에서 이 두 줄을 잠시 빼고 돌려 보니 실제로 `Beta Beat`가 남는 것을 확인**했고, 다시 되돌렸습니다
`refreshList`로 묶음	"받기 → 그리기 → 페이지 줄"이 32, 33, 35, 37번에서 반복되기 때문입니다
`type="search"` 칸의 × 버튼	누르면 `input` 이벤트가 일어나서 칸을 비운 것과 같게 동작합니다 (따로 처리하지 않습니다)

6. 시험 (실제 myProject.html, 실제 서버, 6개 + 앞의 시험 다시)

프로젝트 4개(`Alpha Song`, `Beta Beat`, `Gamma Loop`, `alpine`)를 만들고, 글자 입력과 이벤트를 페이지 안에서 일으켰습니다. `window.fetch`를 감싸서 목록 요청 수를 셌습니다.

#	한 일	결과
01	`alp`를 0.06초 간격으로 입력	치는 동안 요청은 0번이고, 멈춘 뒤 **1번** 가며, `alpine, Alpha Song`, `?query=alp`가 됩니다
02	공백 하나를 더 입력	요청 0번
03	정렬을 Name으로	바로 1번 가고, `Alpha Song, alpine`, `?query=alp&sort=title`이 됩니다
04	검색 칸 비움	1번 가고, 4개, `?sort=title`이 됩니다
05	`gam` 입력 후 바로 Enter	0.15초 안에 `Gamma Loop`가 나오고, 끝까지 요청은 **1번**입니다 (타이머가 취소되었습니다)
06	`beta` 요청이 1.2초 늦게 도착하게 하고, 그사이 `alpine` + Enter	화면과 주소가 `alpine` 그대로입니다 (옛 응답을 버렸습니다). 가드를 빼면 `Beta Beat`로 덮어씌워집니다

앞의 시험 다시 (30번과 32번을 고쳤으므로): 30번의 14개와 32번의 10개가 모두 그대로 통과했습니다. 페이지 넘김 클릭은 여전히 요청 1번이고 `?page=2`가 됩니다.